UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ

Ing. Tecnologías de la Información e Innovación Digital

Asignatura: Optativa I. Bases de Datos en la Nube

Profesor: M.T.I. Marco Antonio Ramírez Hernández

Grado y Grupo: 7° B

PROYECTO

Práctica 03 — Manipulación de datos, relación N:M, procedimientos, funciones, eventos y vista

---

Presentado por:

1. **Obed Vargas Luna** — Matricula: 240145

1. Objetivo

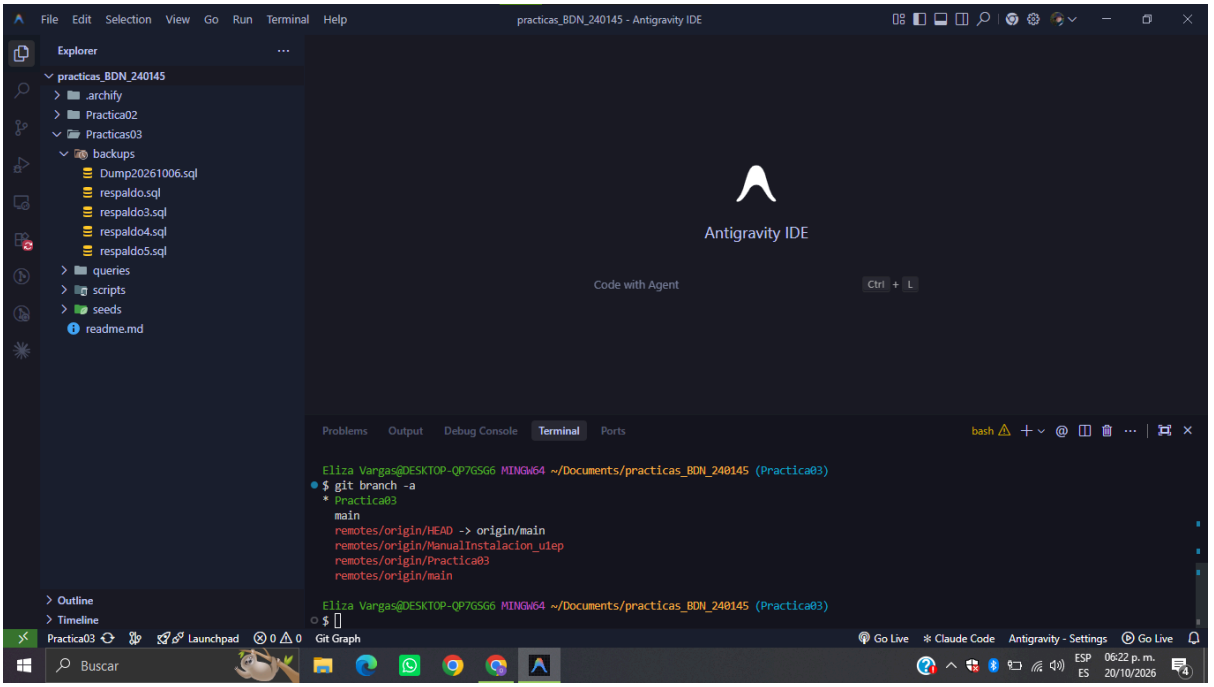
Documentar el procedimiento completo para configurar localmente un entorno con **PostgreSQL 15** y **pgAdmin 4** en Visual Studio Code, subir la configuración a **GitHub** mediante un *Personal Access Token (PAT)*, conectarse a un servidor **VPS** mediante **PuTTY** (o terminal) y desplegar los contenedores para acceder a la base de datos a través de la interfaz web.

2. Configuración Local del Proyecto en Visual Studio Code

1. **Creación de la carpeta del proyecto:**
   Abre Visual Studio Code y selecciona **File > Open Folder**. Crea y selecciona una carpeta para el proyecto (por ejemplo, docker-compose-host).

   En nuestro caso no hicimos eso, porque ya tenemos un repositorio al cual solo se le creo una nueva rama llamada ManualInstalacion_u1ep, como se muestra en la siguiente captura de pantalla por medio del comando

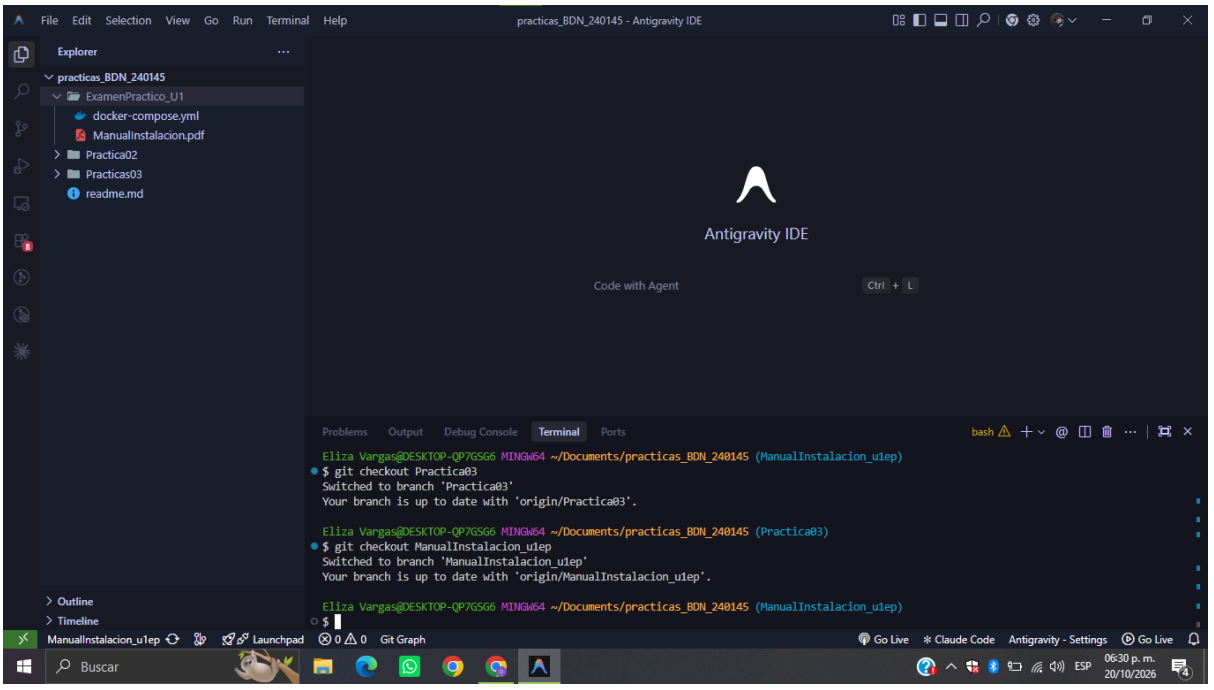
   git branch -a:



2. **Creación del archivo docker-compose.yml:**
   Crea un nuevo archivo en la raíz del proyecto nombrado exactamente docker-compose.yml.

   Nuevamente en nuestro caso particular tenemos la rama antes mencionada, en la cual dentro de la misma tenemos que crear dos archivos dentro de una carpeta, la carpeta se llama ExamenPractico_U1, y dentro de esa carpeta se crean los dos archivos mencionados anteriormente los cuales son, docker-compose.yml (este es el entorno donde se configura la imagen) y manualInstalacion.pdf (este es el pdf del Manual de instalación el cual es este).

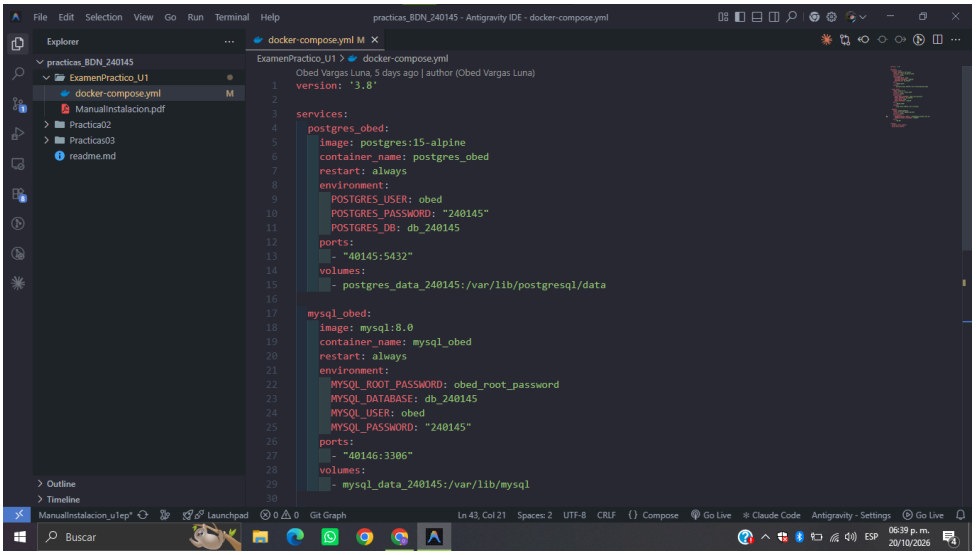
   Como se muestra en la siguiente captura de pantalla:

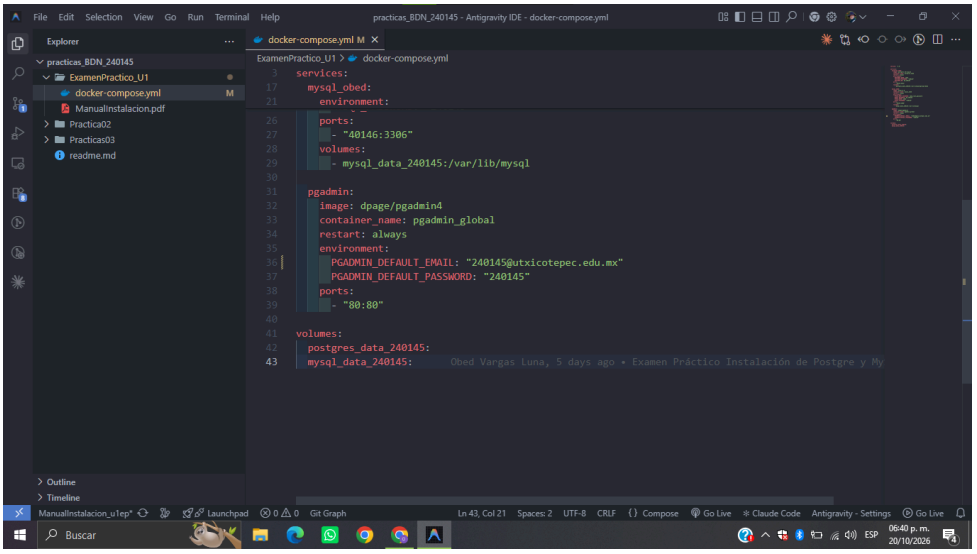


3. **Estructura de la configuración:**
   Agrega la siguiente definición de servicios (reemplaza `240099` por tu matrícula y `40099` por el puerto asignado)

   En el archivo `docker-compose.yml`, definí la configuración de los tres servicios requeridos (**PostgreSQL 15**, **MySQL 8.0** y **pgAdmin 4**), adaptando mis parámetros personales con mi usuario `obed`, mi matrícula `240145` y los puertos que me fueron asignados (`40145` para PostgreSQL, `40146` para MySQL y `80` para pgAdmin):

3.1. Código completo de mi archivo `docker-compose.yml`





3.2. Explicación detallada de los parámetros por servicio

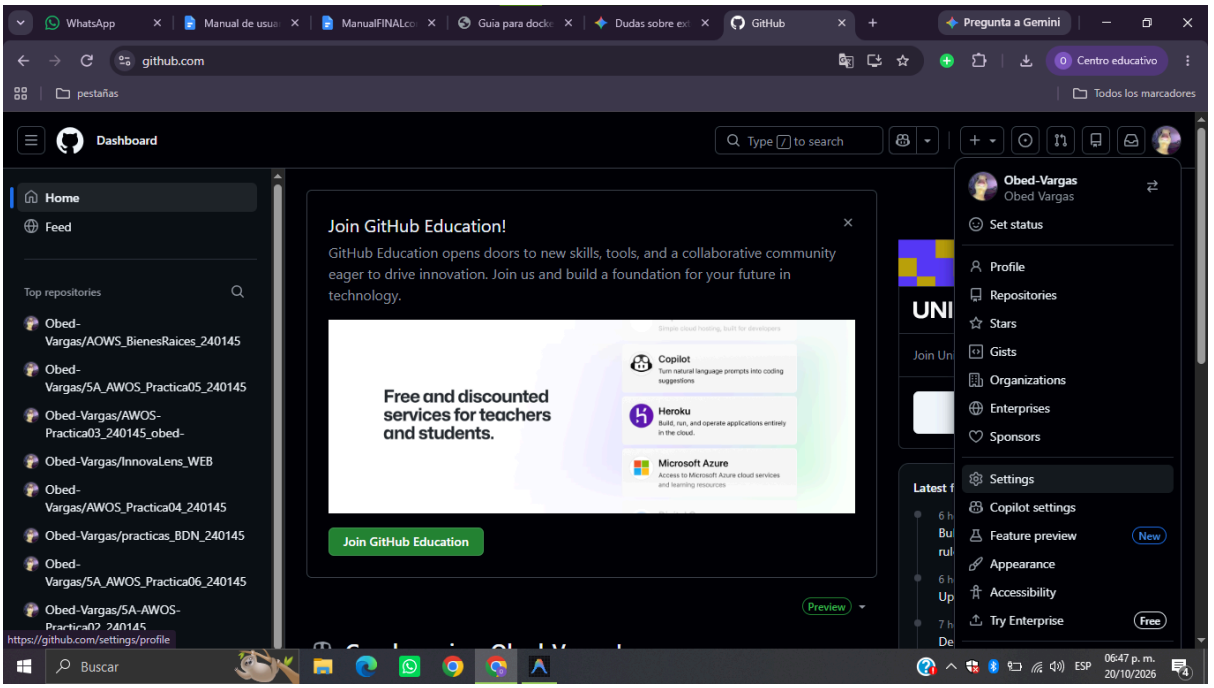
- **Servicio de PostgreSQL (`postgres_obed`):**
  - **`image: postgres:15-alpine`**: Indiqué que se descargue la versión 15 de PostgreSQL basada en la distribución liviana Alpine Linux.
  - **`container_name: postgres_obed`**: Asigné un nombre personalizado al contenedor para identificarlo fácilmente en Docker.
  - **`restart: always`**: Establecí que el contenedor se mantenga encendido o se reinicie automáticamente ante caídas del sistema o del servidor.
  - **`environment`**:
    - `POSTGRES_USER: obed`: Asigné mi nombre como el superusuario de la base de datos.
    - `POSTGRES_PASSWORD: "240145"`: Establecí mi matrícula como contraseña de acceso.
    - `POSTGRES_DB: db_240145`: Definí el nombre de la base de datos que se creará por defecto al iniciar el servicio.
  - **`ports` (`"40145:5432"`)**: Enlacé el puerto `40145` de mi VPS con el puerto interno `5432` del contenedor de PostgreSQL.
  - **`volumes`**: Enlacé el volumen persistente `postgres_data_240145` con la ruta `/var/lib/postgresql/data` dentro del contenedor para no perder la información guardada.
- **Servicio de MySQL (`mysql_obed`):**
  - **`image: mysql:8.0`**: Seleccioné la imagen oficial del motor de base de datos MySQL en su versión 8.0.
  - **`container_name: mysql_obed`**: Definí el nombre del contenedor correspondiente al motor MySQL.
  - **`restart: always`**: Configuré la política para mantener activo el servicio continuamente.
  - **`environment`**:
    - `MYSQL_ROOT_PASSWORD: obed_root_password`: Asigné la contraseña para el usuario administrador supremo (`root`).
    - `MYSQL_DATABASE: db_240145`: Indiqué la base de datos a instanciar al iniciar el contenedor.
    - `MYSQL_USER: obed` y `MYSQL_PASSWORD: "240145"`: Creé mi usuario de trabajo individual vinculado directamente a mi base de datos `db_240145`.

    - **`ports` (`"40146:3306"`)**: Redireccioné el puerto público `40146` asignado en el servidor hacia el puerto estándar `3306` interno de MySQL.
    - **`volumes`**: Asocié el volumen `mysql_data_240145` con la ruta interna `/var/lib/mysql`.
- **Servicio de la Interfaz Web (`pgadmin`):**
    - **`image: dpage/pgadmin4`**: Declaré la imagen oficial del administrador gráfico pgAdmin 4.
    - **`container_name: pgadmin_global`**: Asigné el nombre identificador para el contenedor de la interfaz de administración.
    - **`restart: always`**: Aseguré el reinicio automático del servicio.
    - **`environment`**:
        - `PGADMIN_DEFAULT_EMAIL: "240145@utxicotepec.edu.mx"`: Configuré mi correo institucional como nombre de usuario para el inicio de sesión.
        - `PGADMIN_DEFAULT_PASSWORD: "240145"`: Establecí mi matrícula como contraseña para ingresar a la plataforma web.
    - **`ports` (`"80:80"`)**: Mapeé directamente el puerto HTTP estándar `80` del servidor con el puerto `80` del contenedor web.
- **Sección de Volúmenes (`volumes`):**
    - **`postgres_data_240145`** y **`mysql_data_240145`**: Definí ambos volúmenes locales independientes para garantizar que la información de mis bases de datos persista dentro del disco del servidor, aun si elimino o vuelvo a levantar los contenedores.

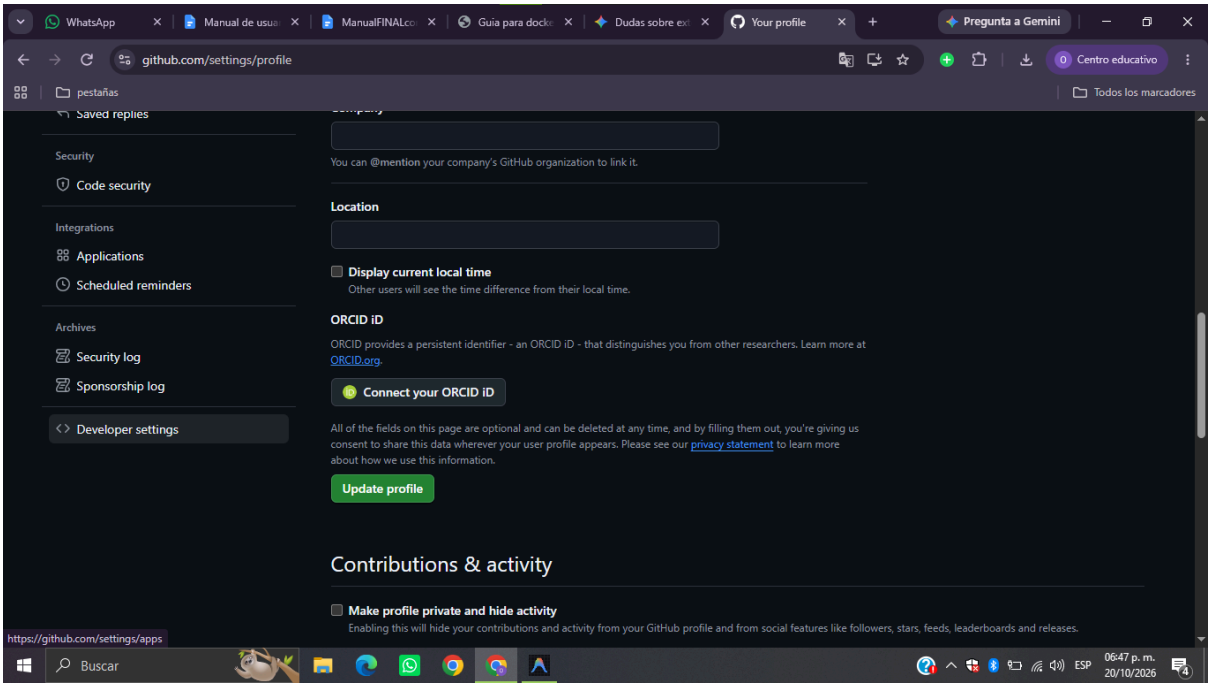
4. Generación del Personal Access Token (PAT) en GitHub

Dado que GitHub requiere autenticación mediante tokens de acceso personal para ejecutar comandos de Git desde la terminal:

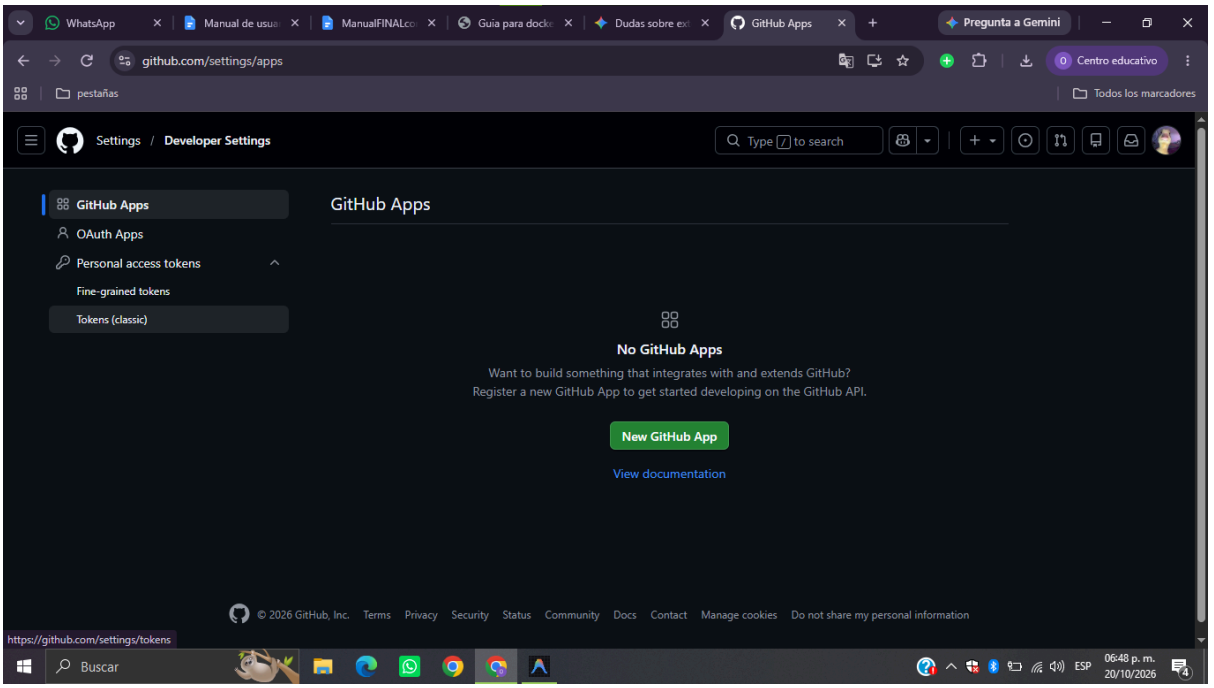
1. **Acceso a la configuración de la cuenta:**
   Inicié sesión en mi cuenta de GitHub, hice clic en mi foto de perfil (esquina superior derecha) y seleccioné **Settings**.

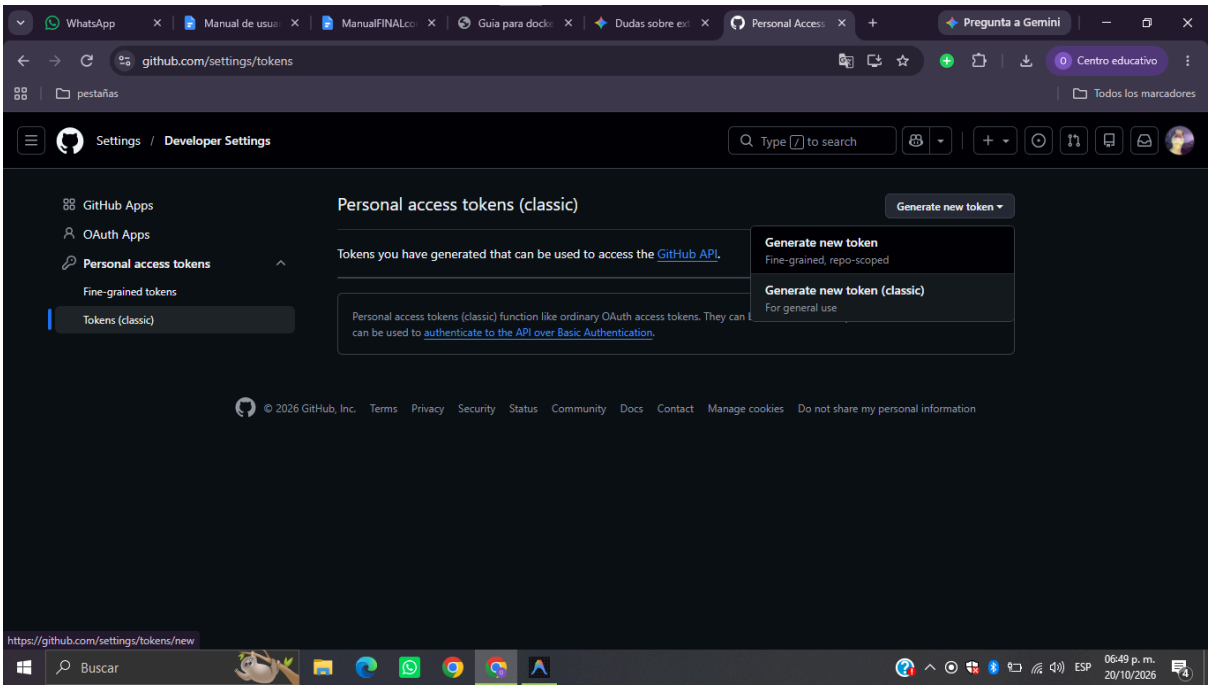


2. **Ingreso a los ajustes de desarrollador:**
   En el menú lateral izquierdo, me desplacé hacia la parte inferior y seleccioné **Developer settings**.

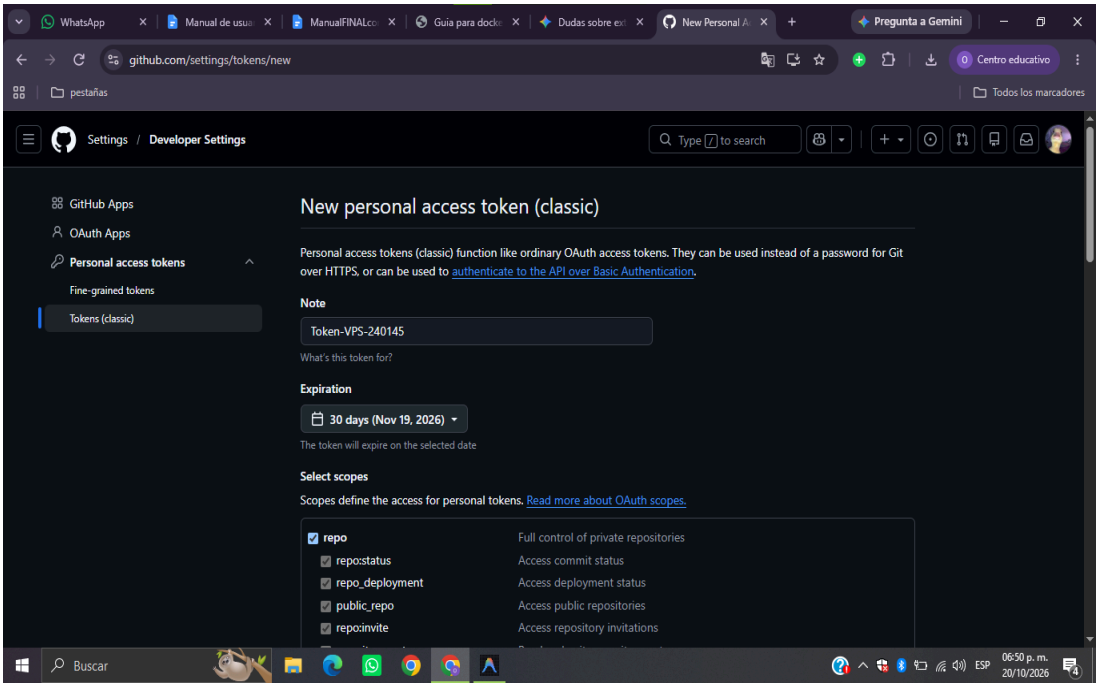


3. **Creación del Token Classic:**
   Navegué a **Personal access tokens > Tokens (classic)** y seleccioné la opción **Generate new token > Generate new token (classic)**.

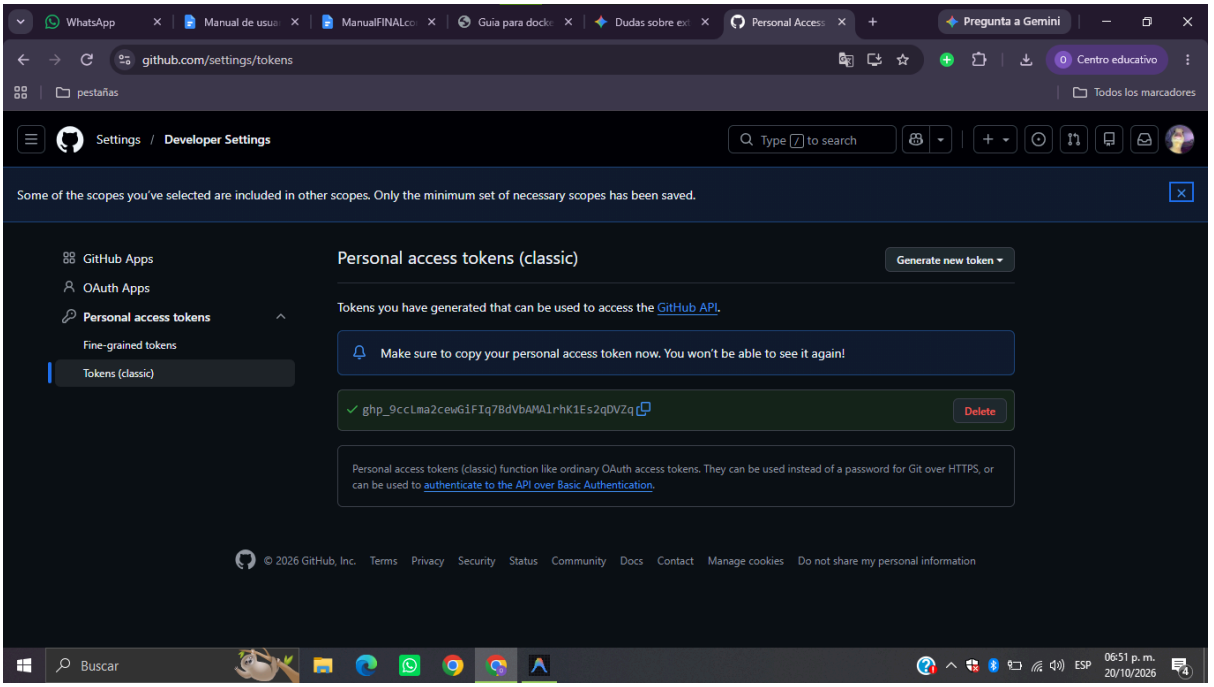




4. **Configuración de los parámetros del token:**
   Completé los campos de la siguiente manera:
   - **Note:** Asigné un identificador para reconocer el token (por ejemplo: Token-VPS-240145).
   - **Expiration:** Seleccioné el tiempo de validez del token (30 días o sin expiración).
   - **Select scopes:** Marqué la casilla principal **repo** para otorgar los permisos necesarios de lectura y escritura sobre mis repositorios.



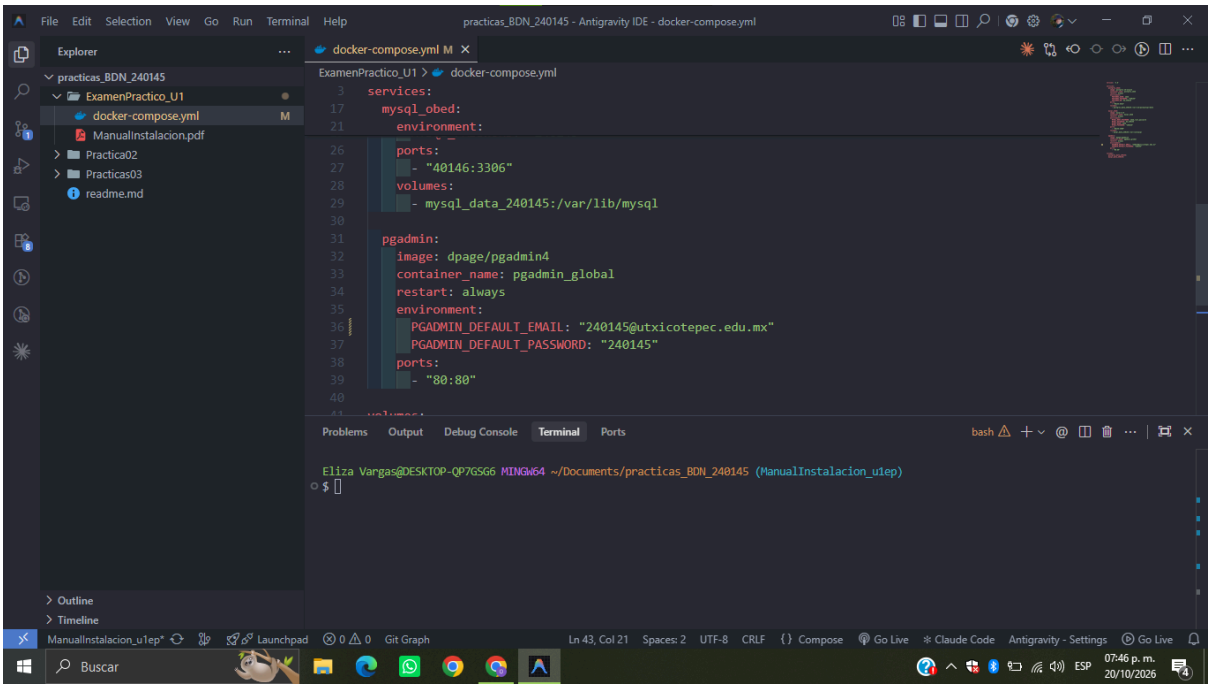
5. **Generación y resguardo del token:**
   Presioné el botón **Generate token** al final del formulario. Copié la clave generada que comienza con ghp_... y la guardé en un lugar seguro, ya que este token es la contraseña que utilizo al ejecutar los comandos.



5. Subir el Proyecto a mi Repositorio de GitHub

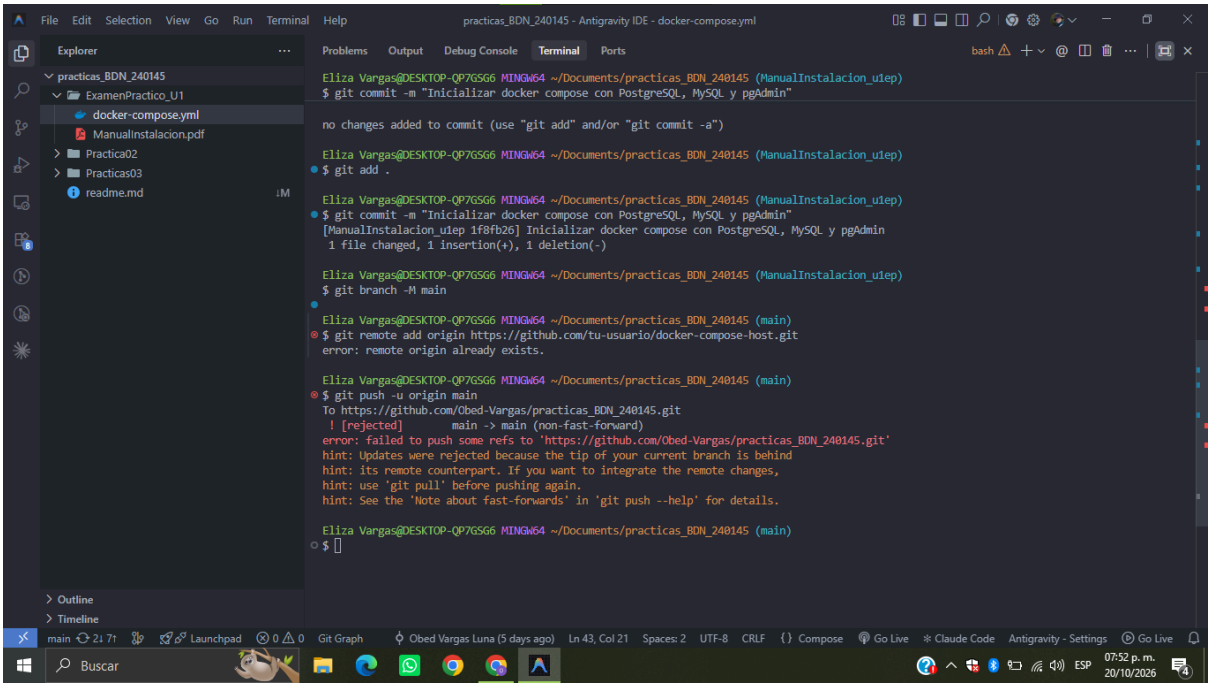
Para respaldar mi archivo docker-compose.yml en el repositorio que ya tenía creado previamente en GitHub y poder descargarlo posteriormente en el servidor VPS:

1. **Apertura de la terminal integrada:**
   De regreso en Visual Studio Code, abrí la terminal integrada presionando la combinación de teclas **Ctrl + Ñ** (o desde el menú **Terminal > New Terminal**).



2. **Vinculación y envío de comandos mediante Git:**
   Ejecuté uno a uno la siguiente serie de comandos para inicializar el repositorio local, agregar mi archivo de configuración, realizar el commit y subir los cambios a mi repositorio existente docker-compose-host:

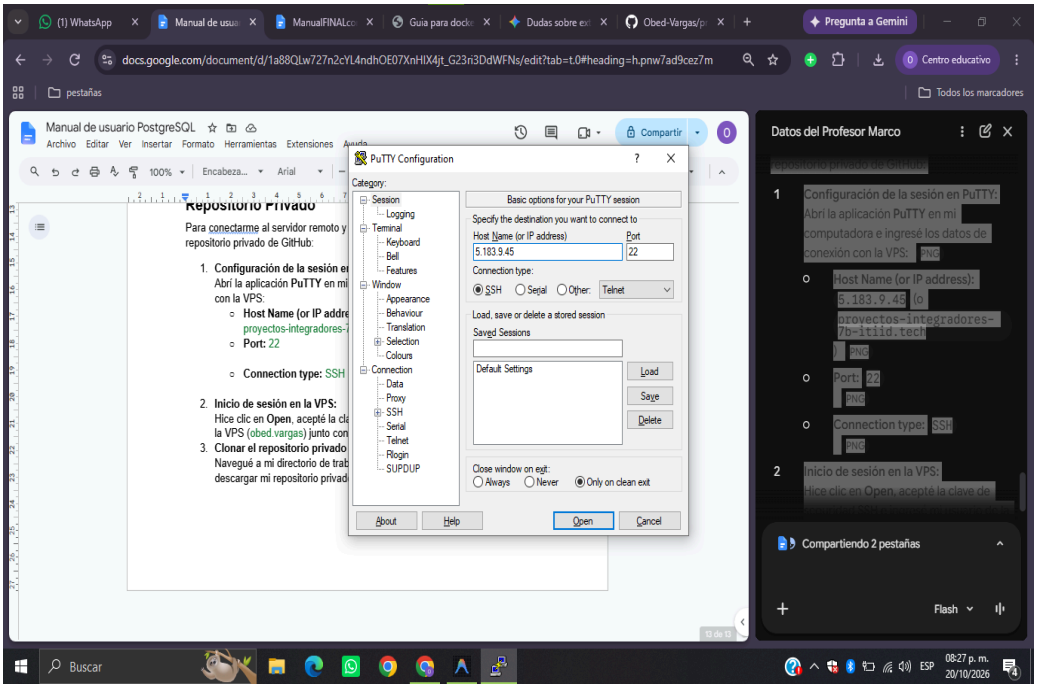
Los comandos son los siguientes (en mi caso no funcionaron porque ya habia hecho un commit y habia pusheado mis cambios de la imagen en clase):



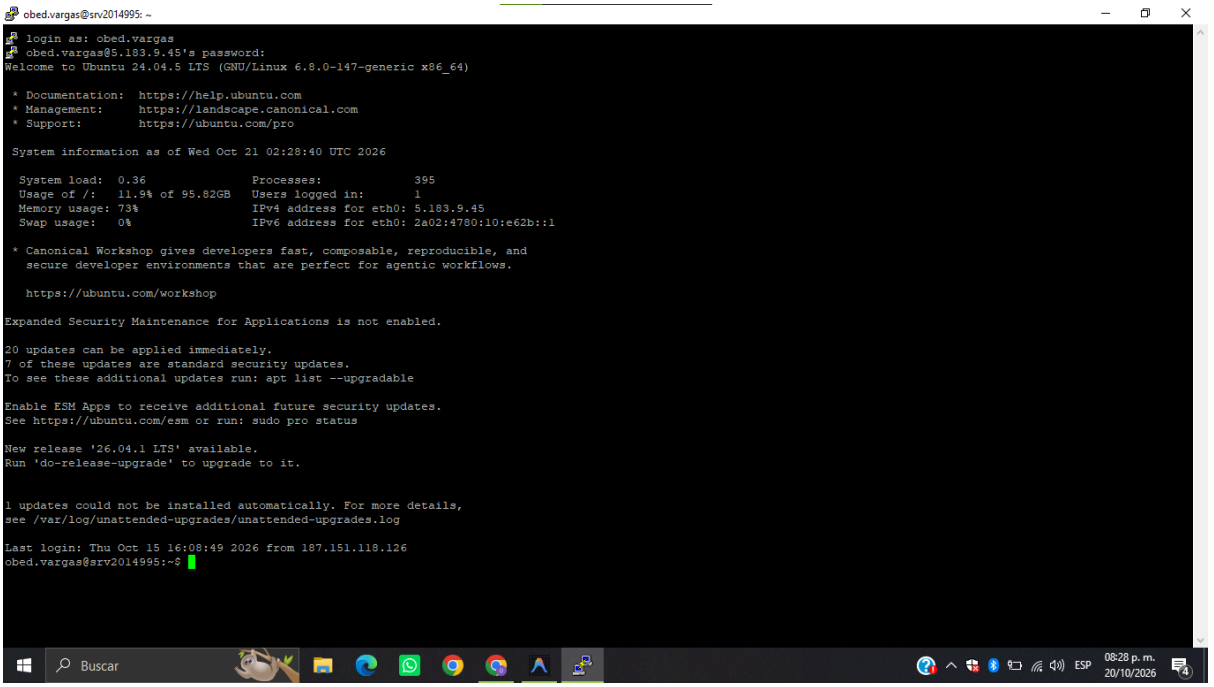
6. Conexión a la VPS con PuTTY y Clonado de Repositorio Privado

Para conectarme al servidor remoto y descargar el proyecto que se encuentra en mi repositorio privado de GitHub:

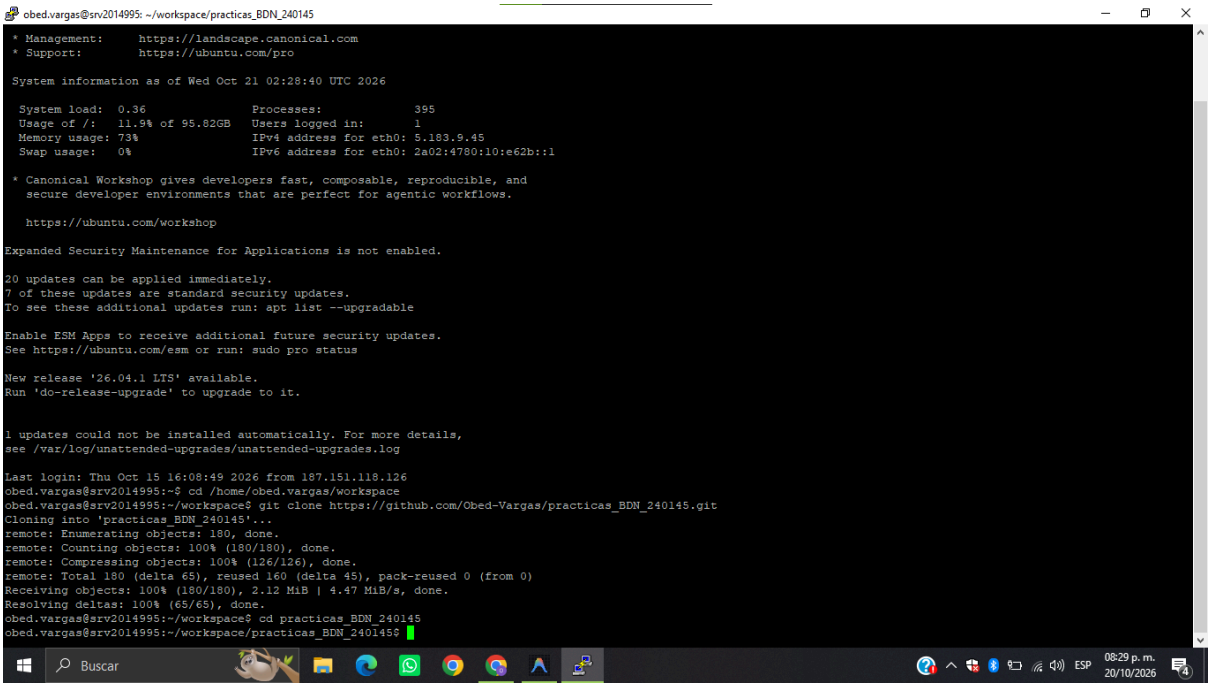
1. **Configuración de la sesión en PuTTY:**
   Abrí la aplicación **PuTTY** en mi computadora e ingresé los datos de conexión con la VPS:
   - **Host Name (or IP address):** 5.183.9.45 (o proyectos-integradores-7b-itiid.tech)
   - **Port:** 22
   - **Connection type:** SSH



2. **Inicio de sesión en la VPS:**
   Hice clic en **Open**, acepté la clave de seguridad SSH e ingresé mi usuario de la VPS (obed.vargas) junto con mi contraseña de acceso.



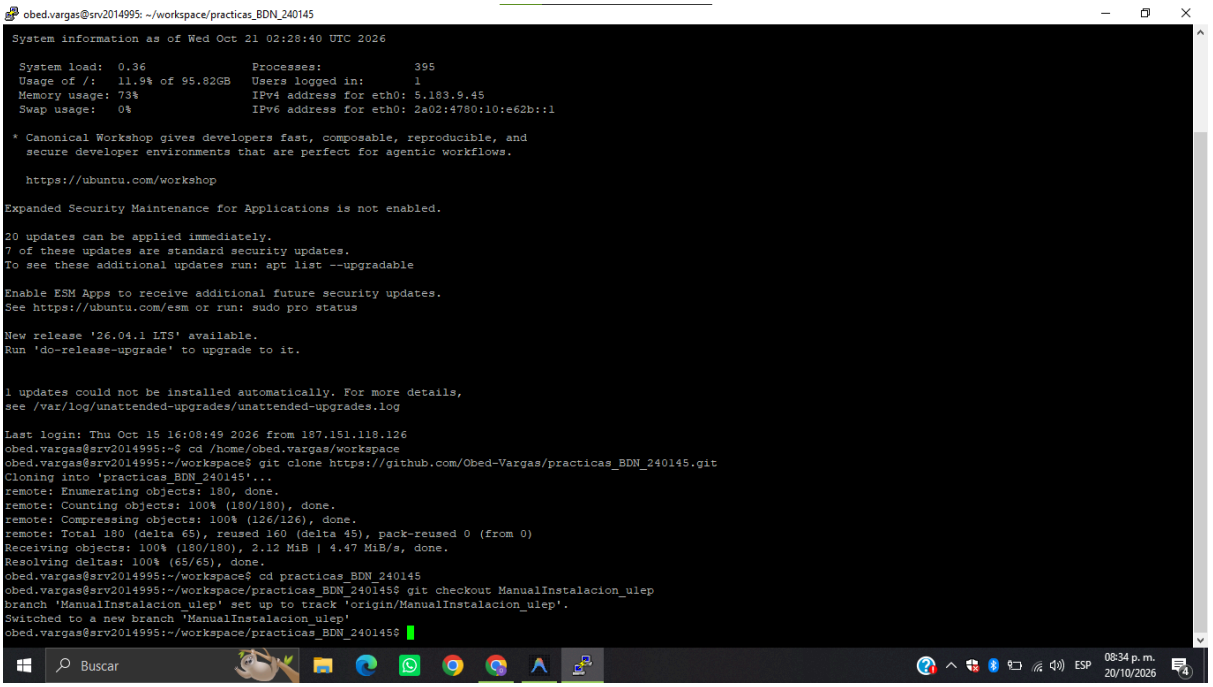
3. **Clonar el repositorio privado en la VPS:**
   Navegué a mi directorio de trabajo y ejecuté el comando git clone para descargar mi repositorio privado:



4. Autenticación con Personal Access Token:
Al tratarse de un **repositorio privado**, la terminal de la VPS me solicitó credenciales de acceso:

- **Username for 'https://github.com':** Ingresé mi usuario Obed-Vargas.
- **Password for 'https://Obed-Vargas@github.com':** Pegué mi Personal Access Token generado anteriormente (ghp_9ccLma2cewGiFIq7BdVbAMAIrhK1Es2qDVZq).

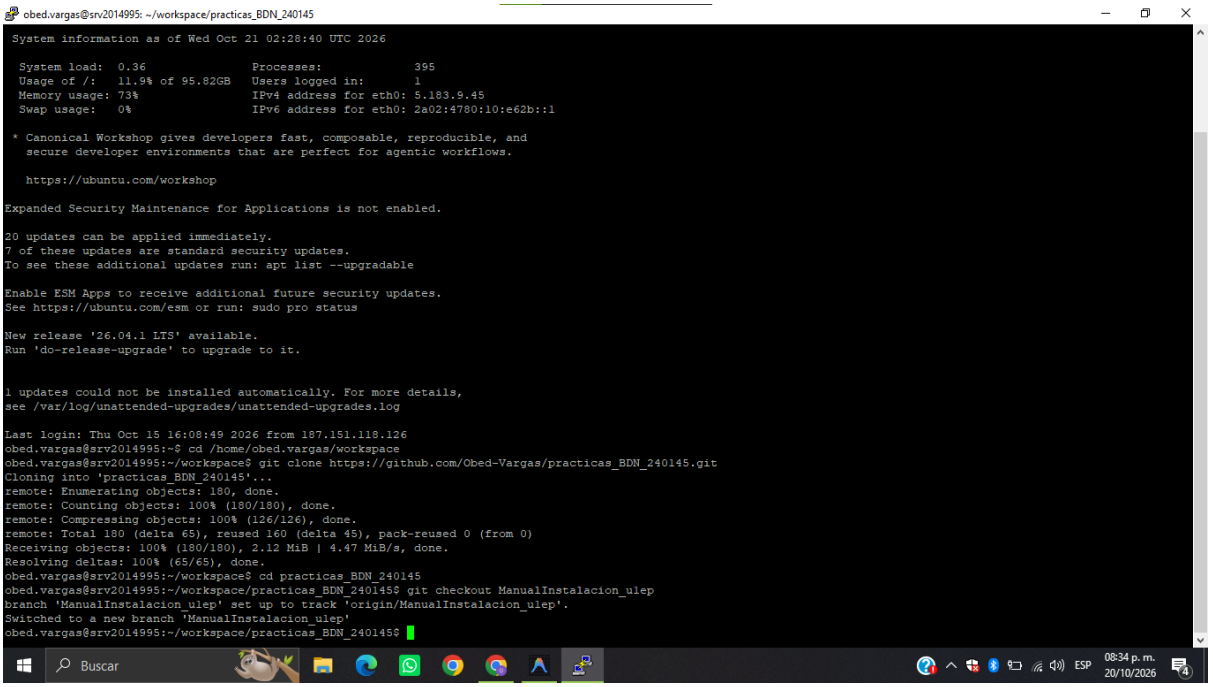
Cambiar a la rama de trabajo:



7. Cambio a mi Rama de Trabajo y Verificación de Archivos

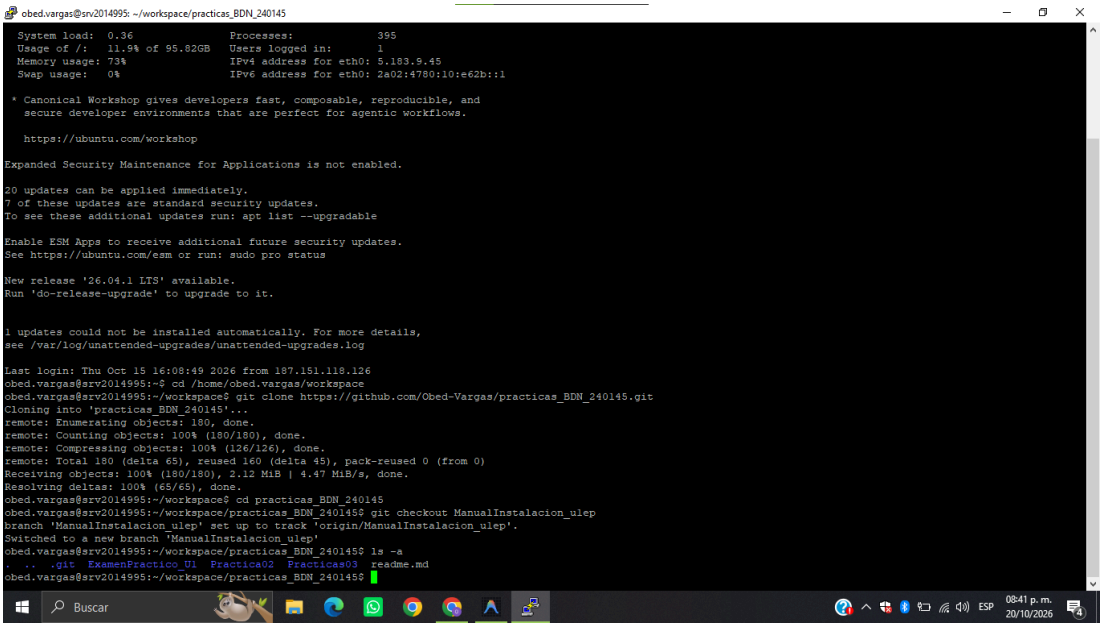
Una vez ubicado dentro del directorio de mi repositorio en la VPS:

1. **Cambiarse a la rama correspondiente:**
   Me cambié a la rama donde subí el archivo de configuración:



Verificar el contenido:

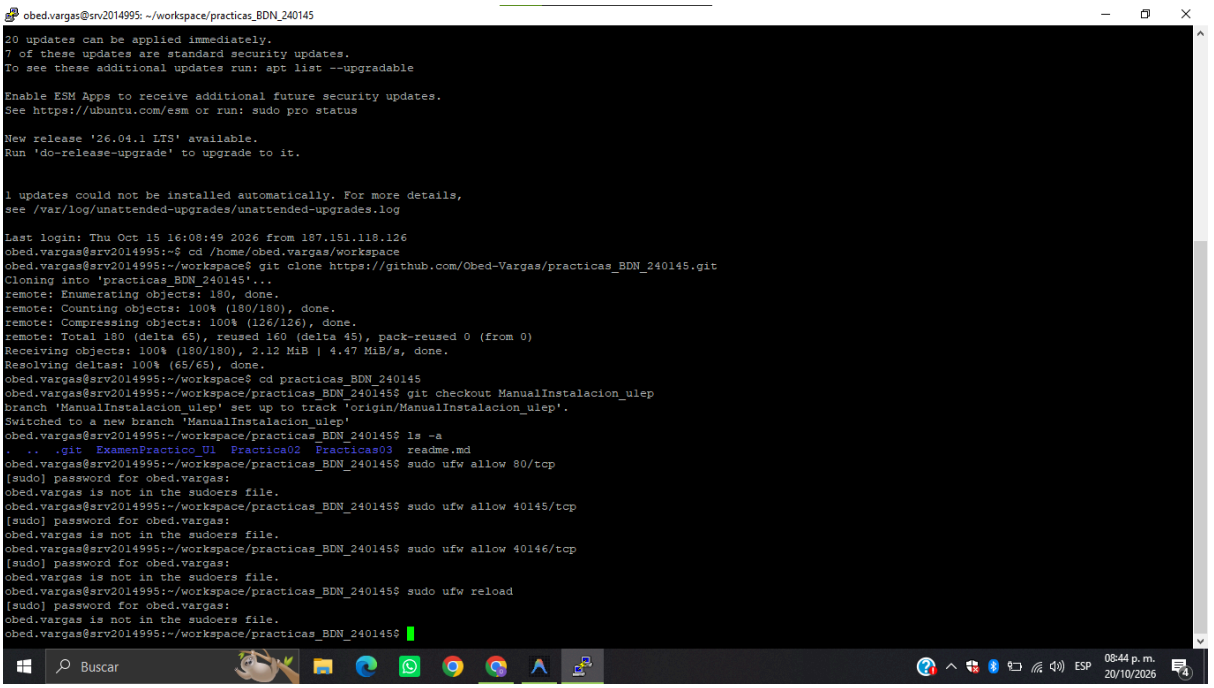
Comprobé que mi archivo `docker-compose.yml` se encontrara presente en el directorio con el comando ls -a:



Habilitación de Puertos en el Firewall y Despliegue de Contenedores

1. ***Autorizar el puerto en el Firewall (`ufw`):***
   Habilité el puerto `80` (correspondiente a mi servicio de pgAdmin) en el firewall de la VPS, con los siguientes comandos:
2. *(Aunque esos comandos no me funcionaron porque evidentemente no tengo permisos)*

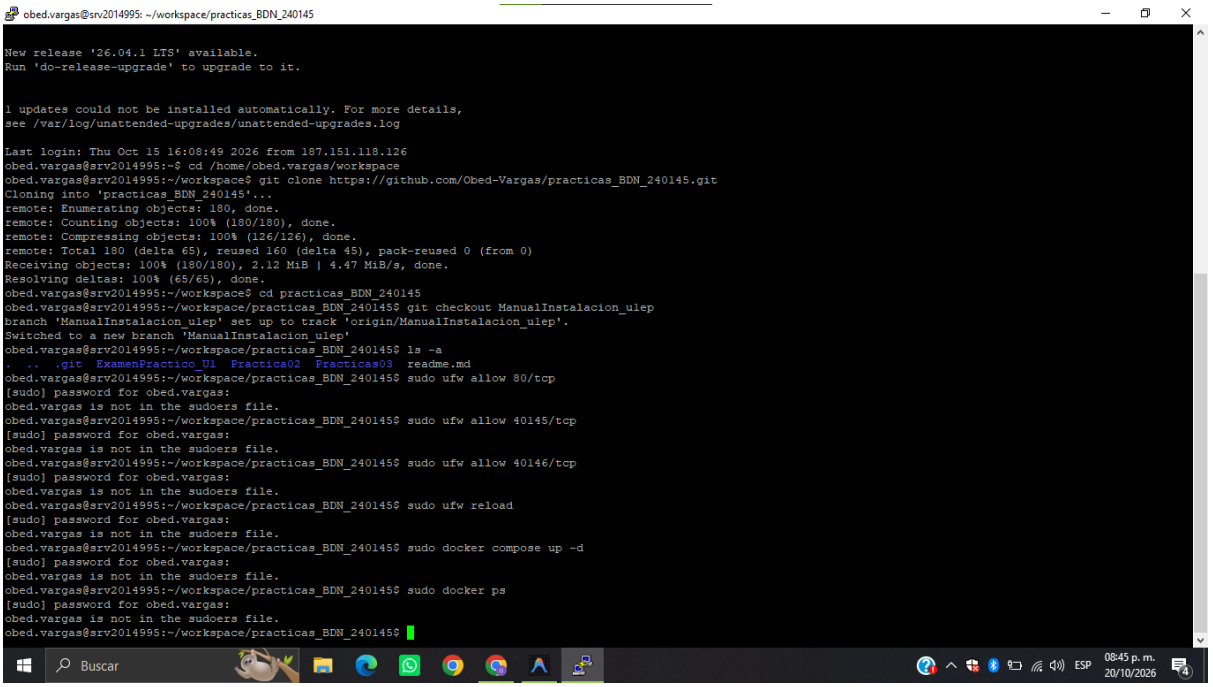
```
sudo ufw allow 80/tcp
sudo ufw allow 40145/tcp
sudo ufw allow 40146/tcp
sudo ufw reload
```



Levantar los servicios con Docker Compose:

Ejecuté el comando para construir y desplegar los contenedores en segundo plano (-d)

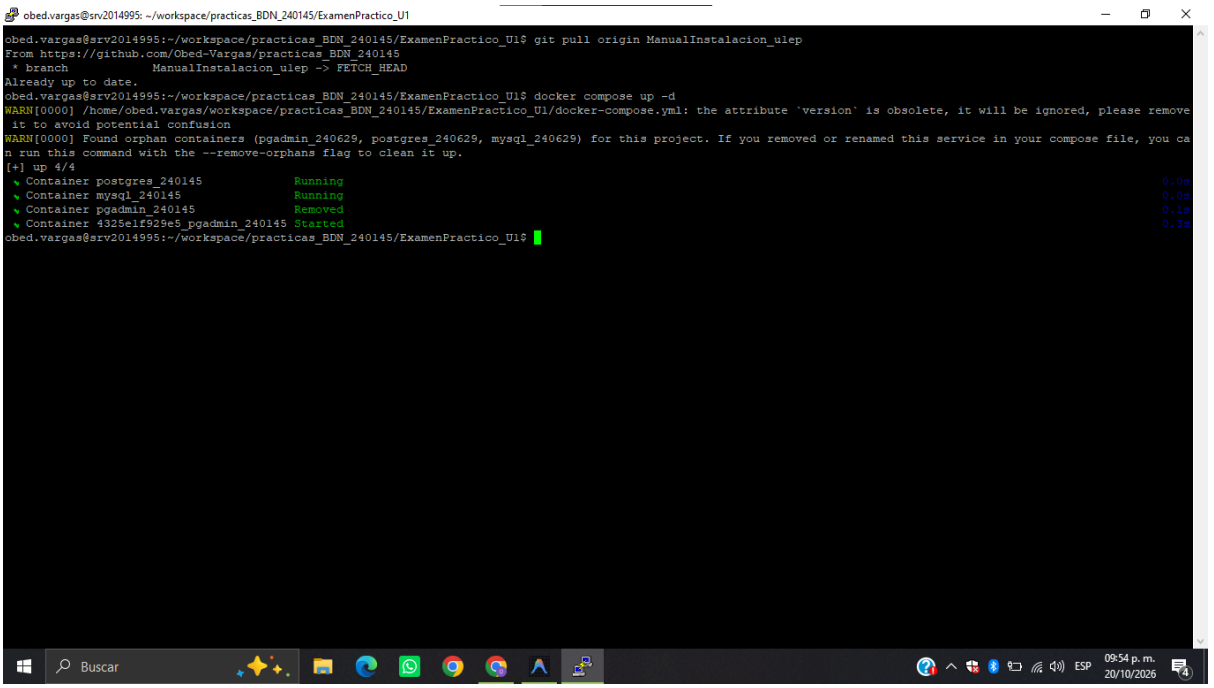
Confirmar estado de ejecución:

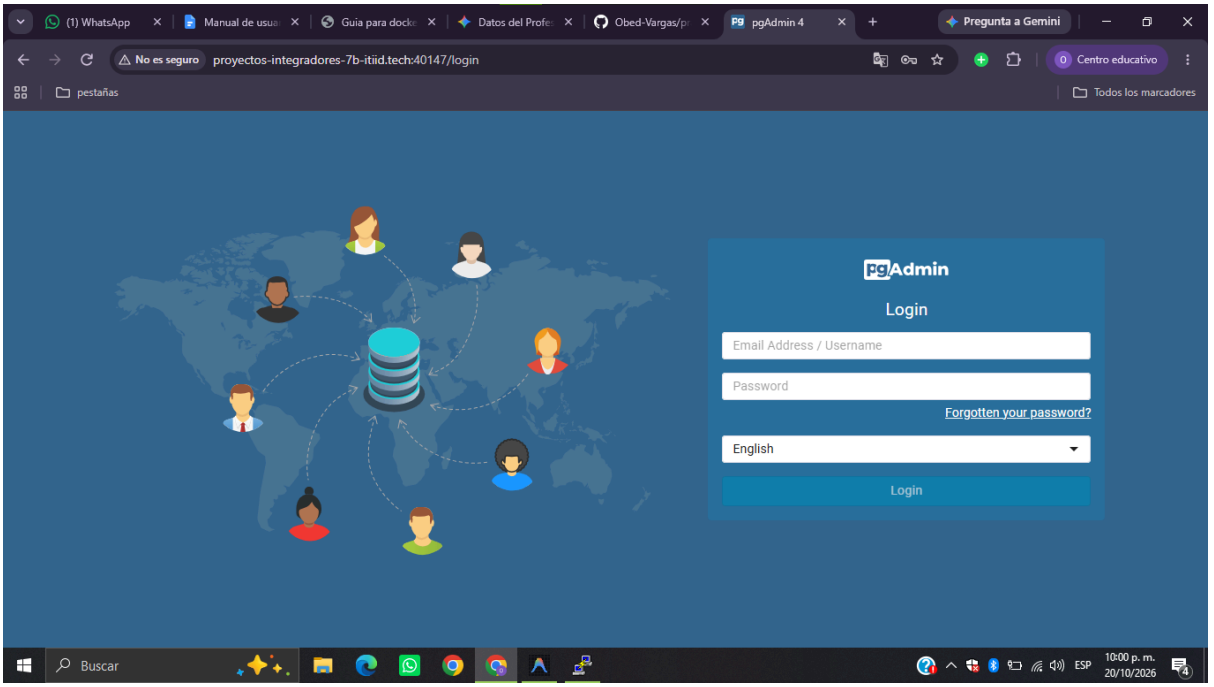


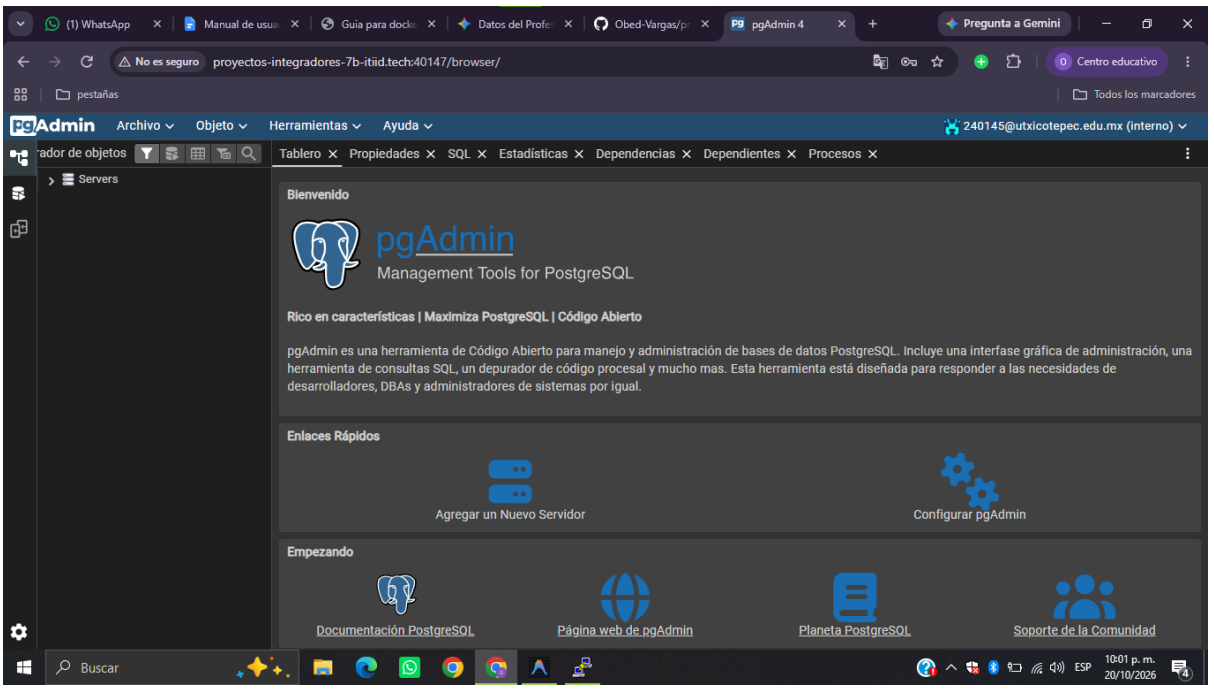
9. Inicio de Sesión y Registro de Base de Datos en pgAdmin

1. ***Acceso web a pgAdmin:***
   Desde el navegador de mi computadora, ingresé a la URL pública:
   http://proyectos-integradores-7b-itiid.tech:40145

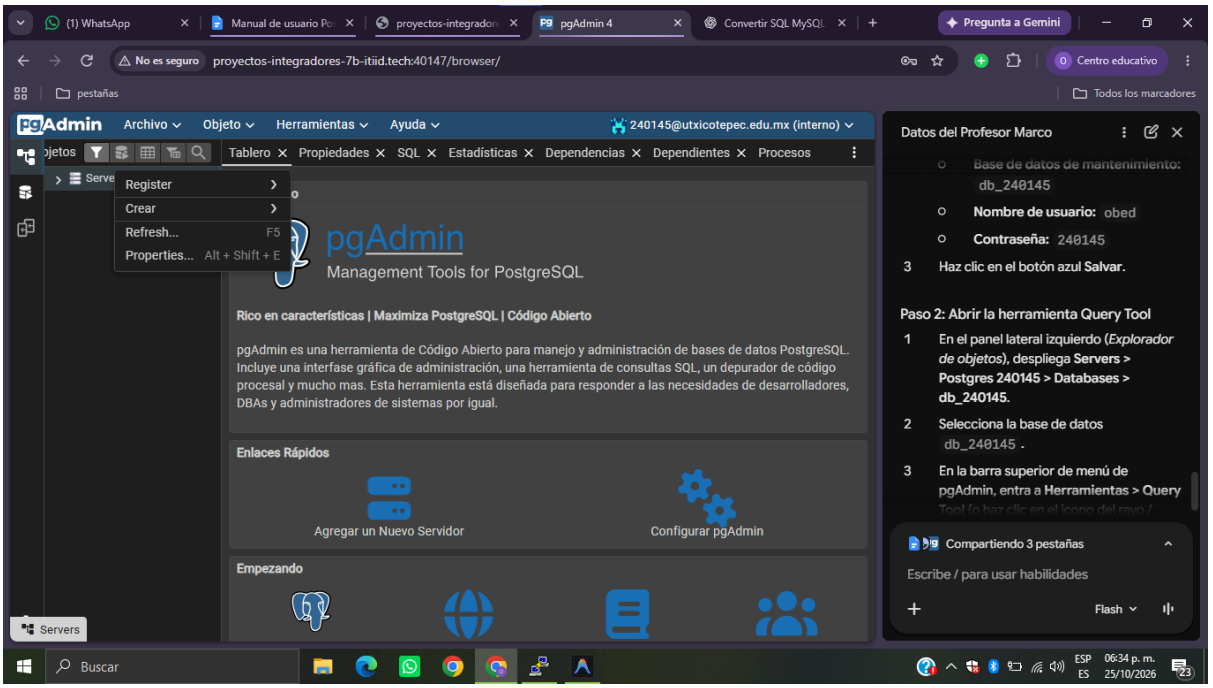
   Para esto debemos primero de habilitar y levantar los servicios con Docker compose:

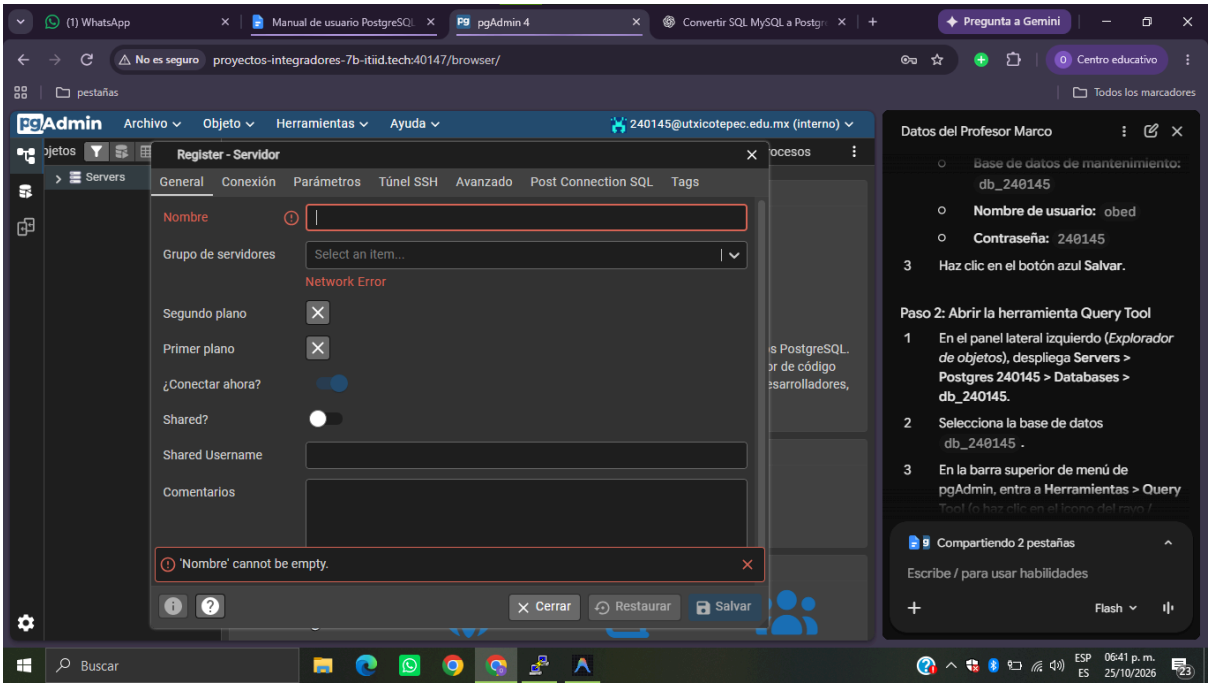



(1) WhatsApp
Manual de usua
Guia para docke
Datos del Profes
Obed-Vargas/p
pgAdmin 4
Pregunta a Gemini
No es seguro
proyectos-integradores-7b-itiid.tech:40147/login
Centro educativo
pestañas
Todos los marcadores
pgAdmin
Login
Email Address / Username
Password
Forgotten your password?
English
Login
Buscar
ESP
10:00 p. m.
20/10/2026

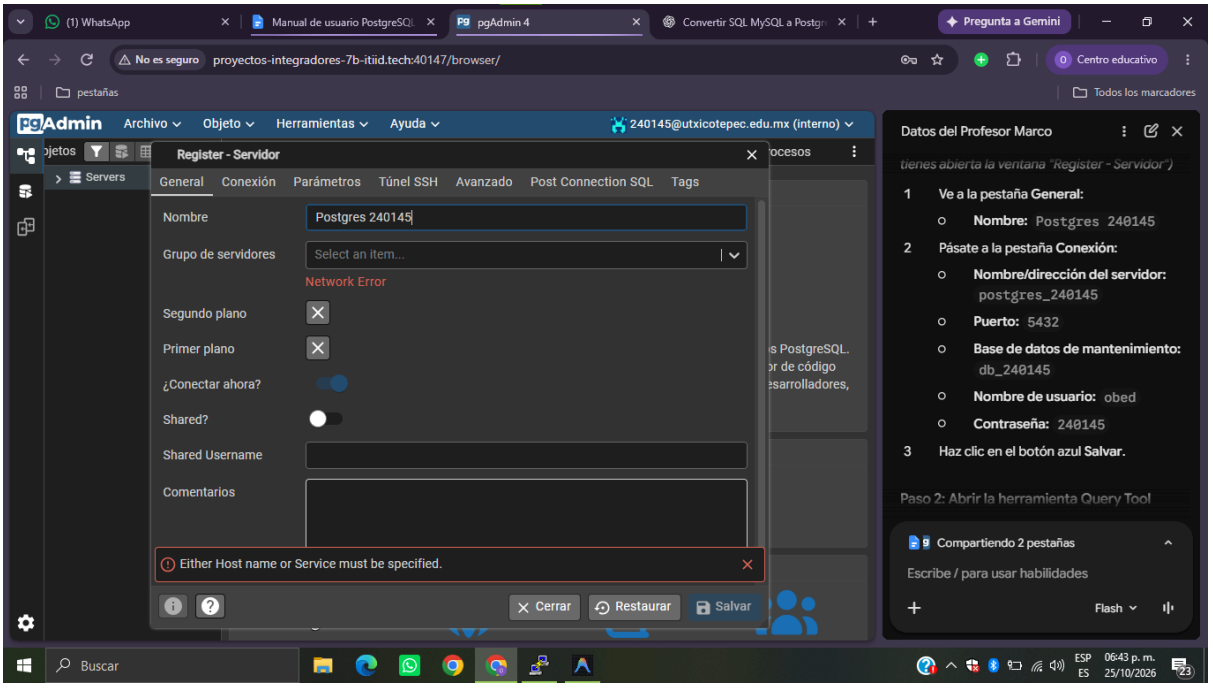

(1) WhatsApp
Manual de usua
Guia para docke
Datos del Profes
Obed-Vargas/p
pgAdmin 4
Pregunta a Gemini
No es seguro
proyectos-integradores-7b-itiid.tech:40147/browser/
Centro educativo
pestañas
Todos los marcadores
pgAdmin
Archivo
Objeto
Herramientas
Ayuda
240145@utxicotepec.edu.mx (interno)
Tablero
Propiedades
SQL
Estadísticas
Dependencias
Dependientes
Procesos
Servers
Bienvenido
pgAdmin
Management Tools for PostgreSQL
Rico en características | Maximiza PostgreSQL | Código Abierto
pgAdmin es una herramienta de Código Abierto para manejo y administración de bases de datos PostgreSQL. Incluye una interfase gráfica de administración, una herramienta de consultas SQL, un depurador de código procesal y mucho mas. Esta herramienta está diseñada para responder a las necesidades de desarrolladores, DBAs y administradores de sistemas por igual.
Enlaces Rápidos
Agregar un Nuevo Servidor
Configurar pgAdmin
Empezando
Documentación PostgreSQL
Página web de pgAdmin
Planeta PostgreSQL
Soporte de la Comunidad
Buscar
ESP
10:01 p. m.
20/10/2026

2. damos click izquierdo a erver y agregamos nuevo server (primero server, registrer y servidor).

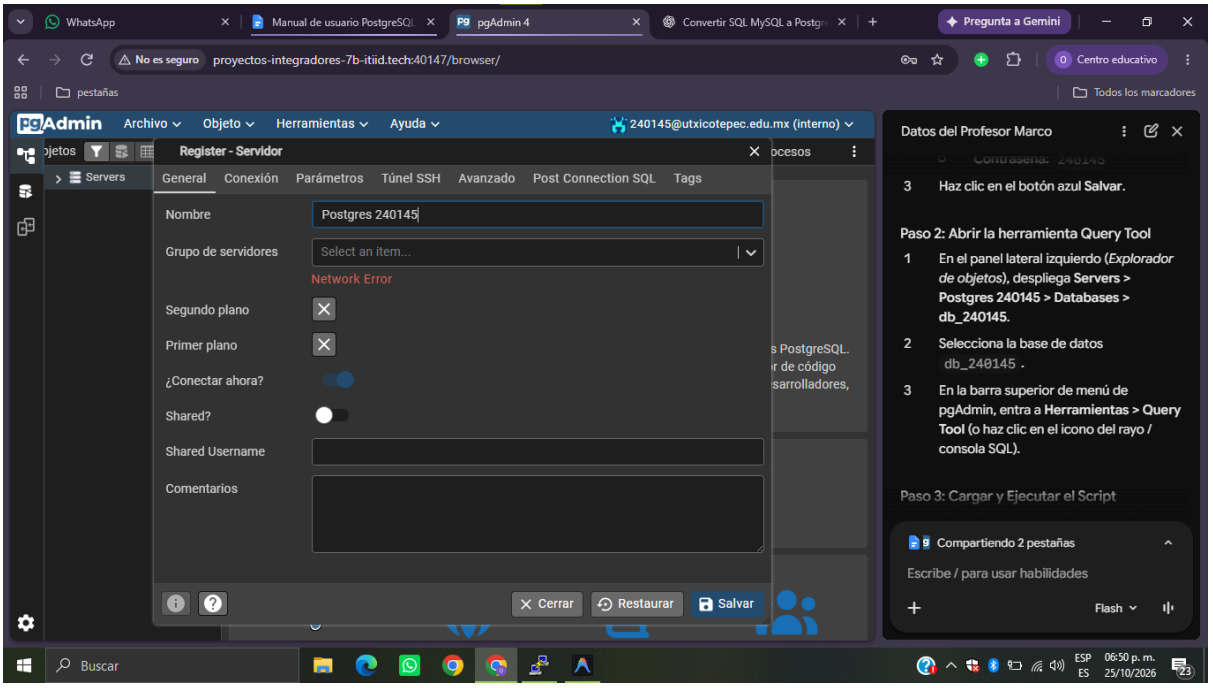


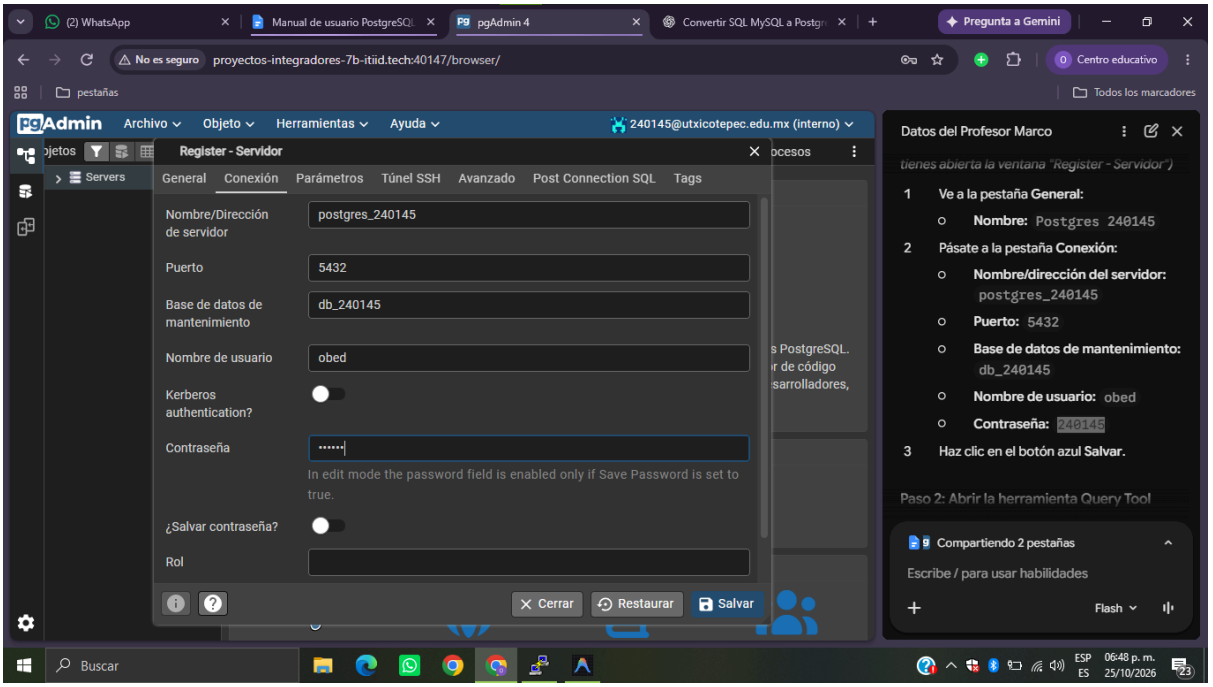


3. Registramos la base de datos y ponemos en general el nombre (ejemplo Postgre 240145).

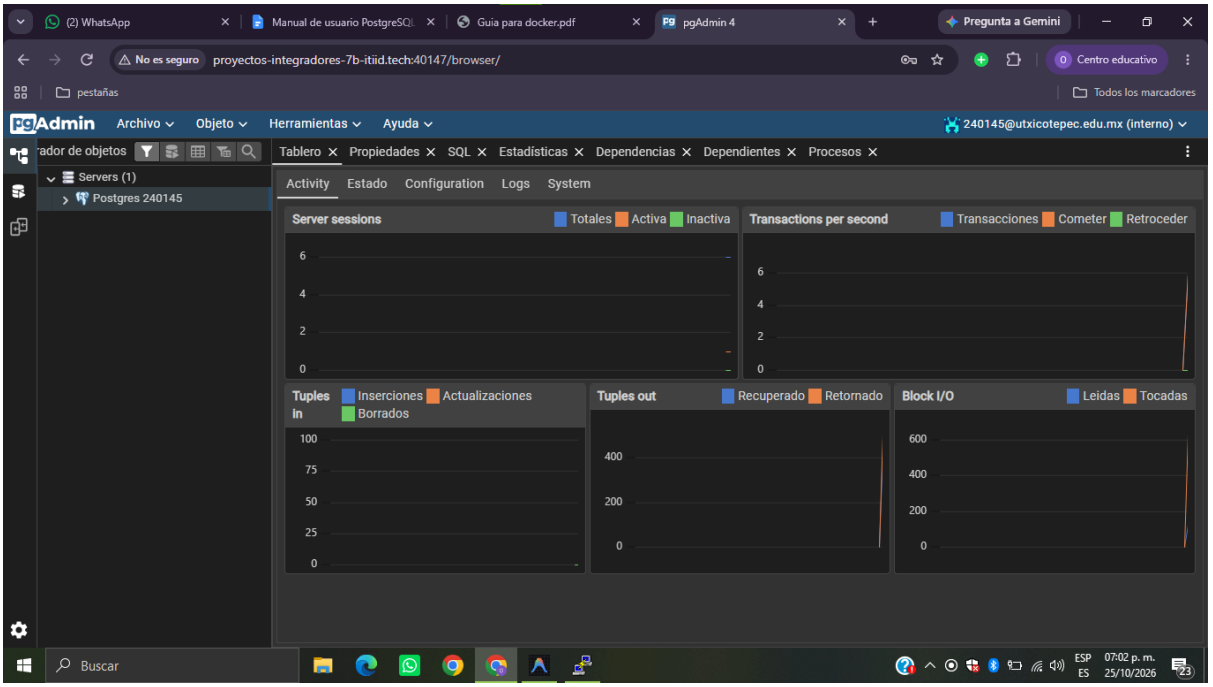


En la parte de conexion ponemos el nombre/dirección del servidor (ej.postgres_240145) y el puerto dejamos el mismo, base de mantenimiento (ej.db_240145), nombre de usuario (ej.Obed), contraseña (ej.240145). y damos en salvar.

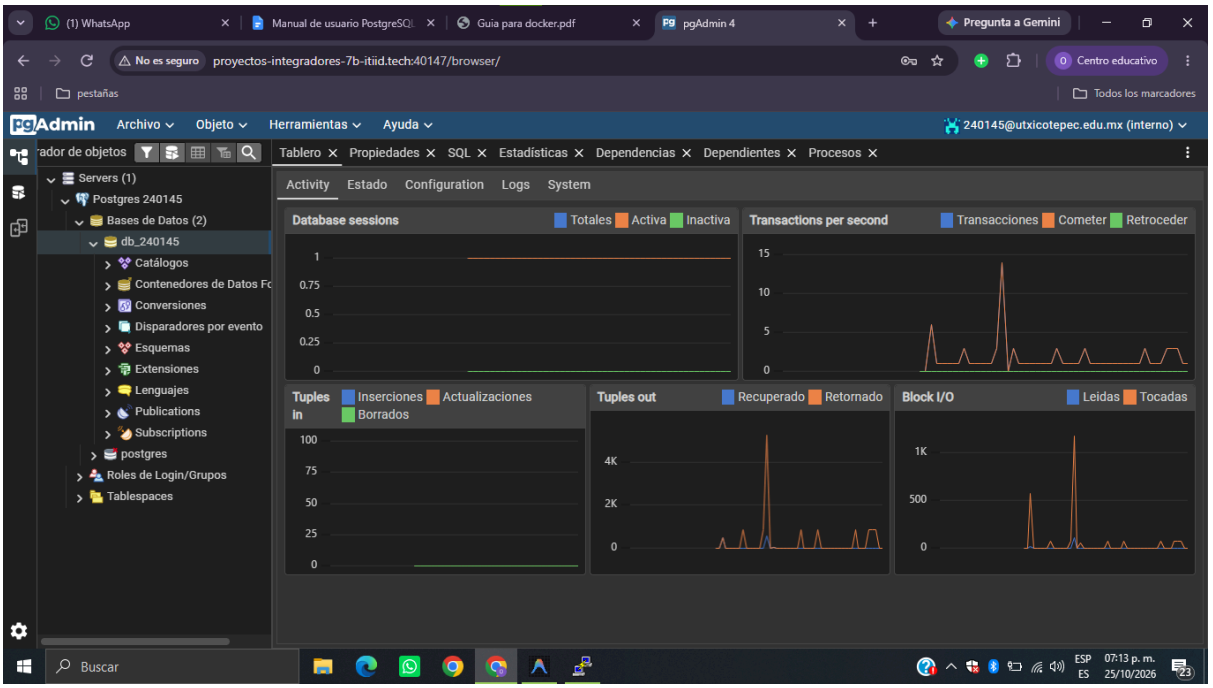




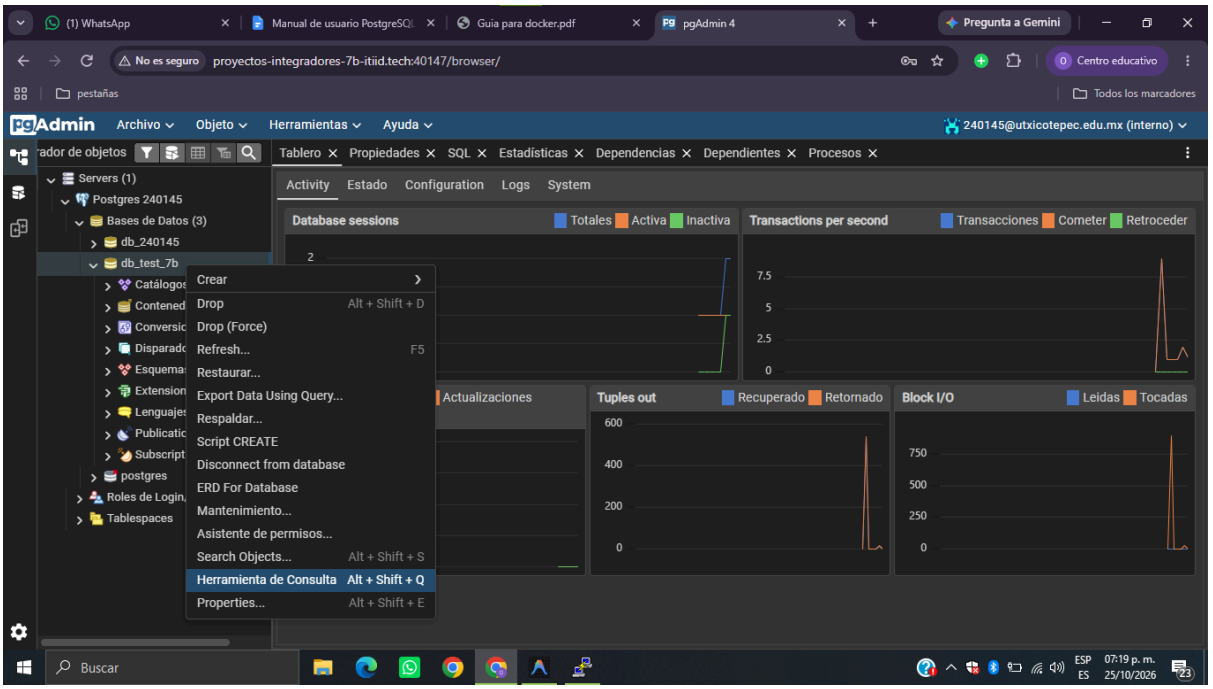
habiendo hecho esto entraremos dentro:



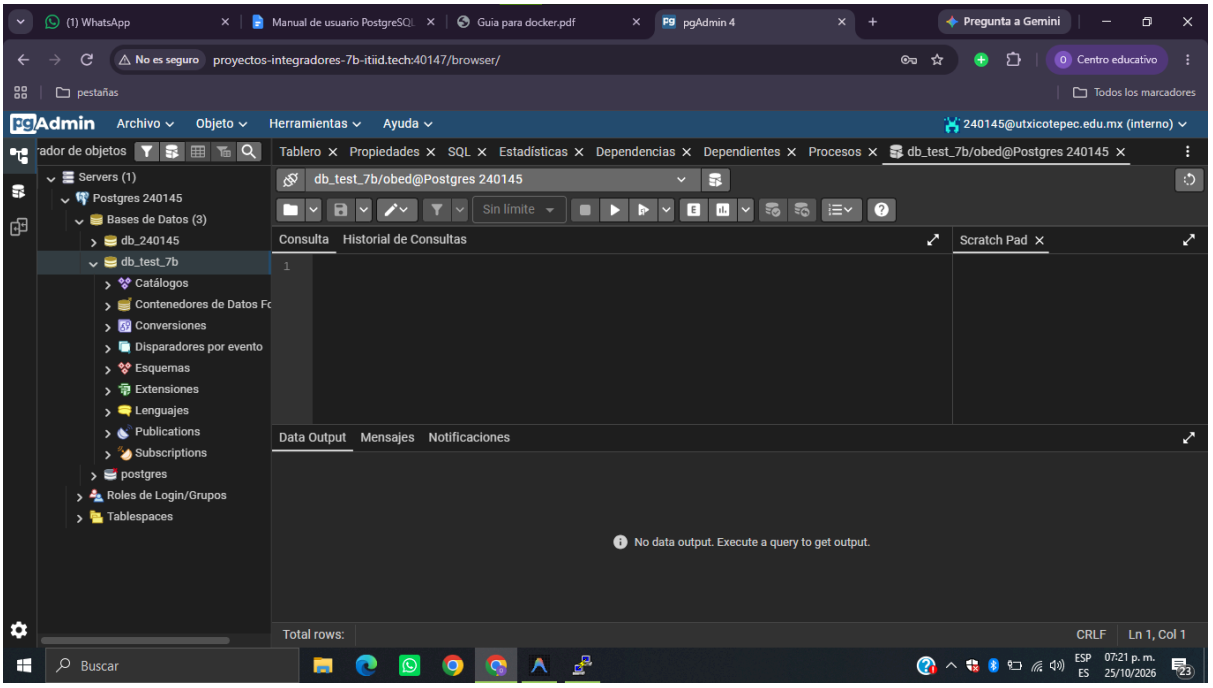
4. Dentro del servidor una vez creado, esra creada nuestra base de datos (se registrara el nombre como la llamamos cuando creamos el server).

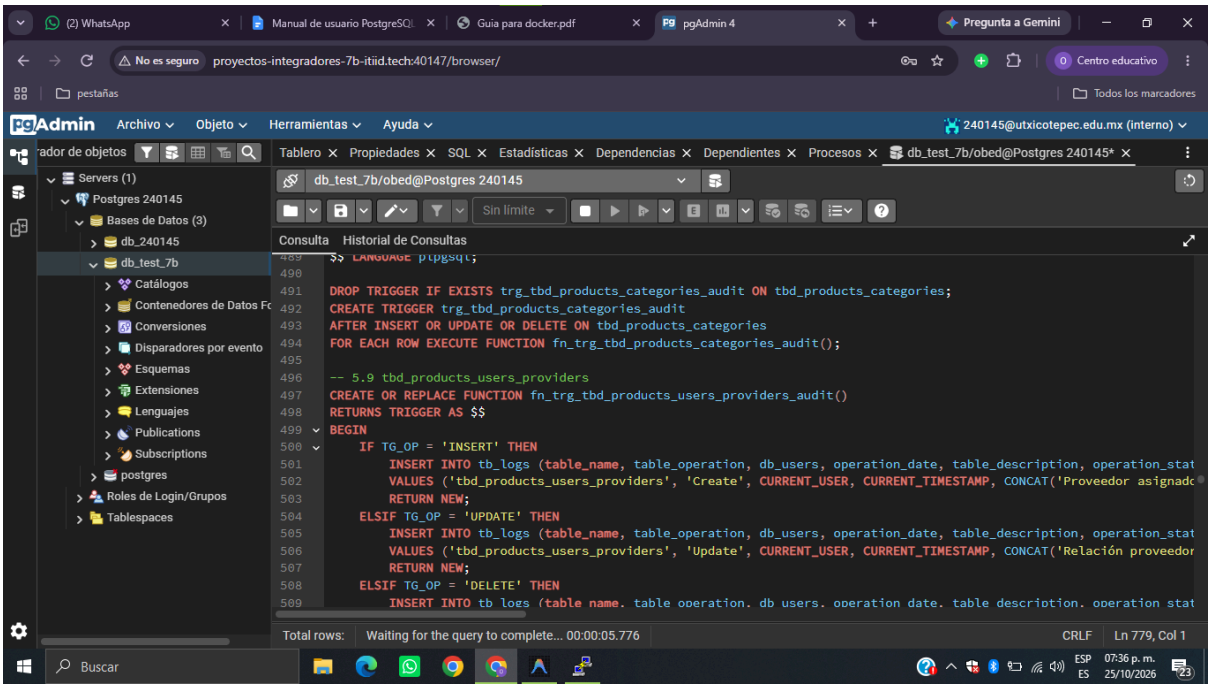


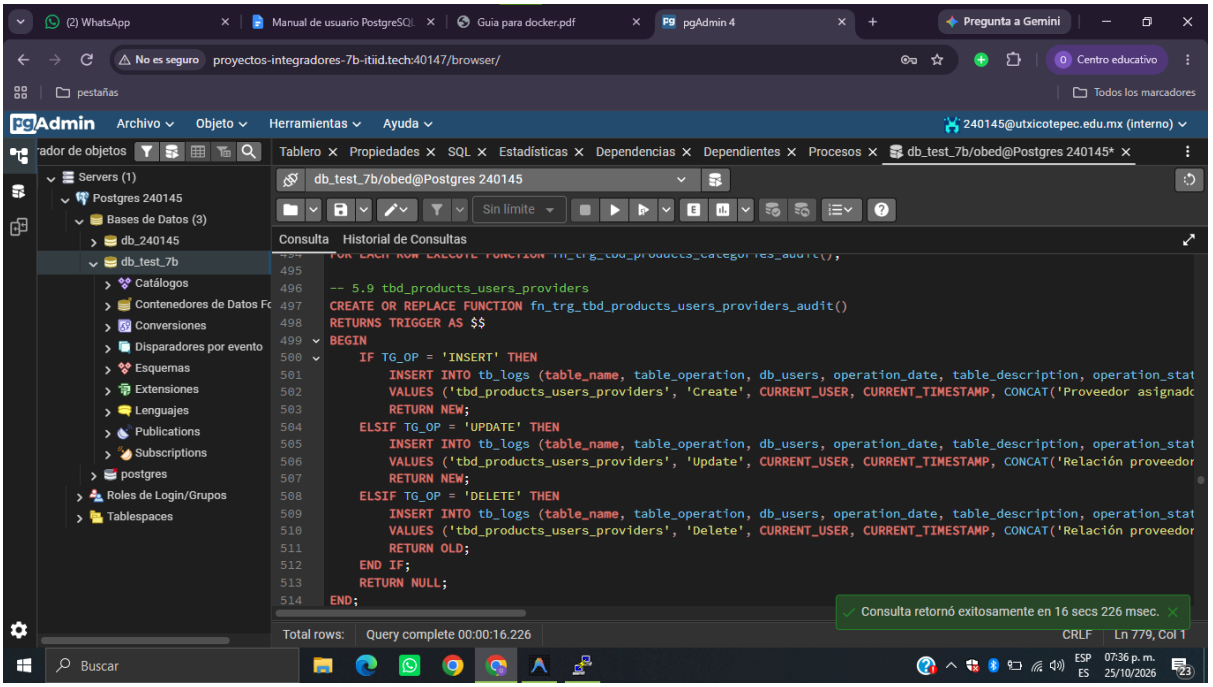
haces click izquierdo en la base de datos, y seleccionamos la opción de herramientas de consulta

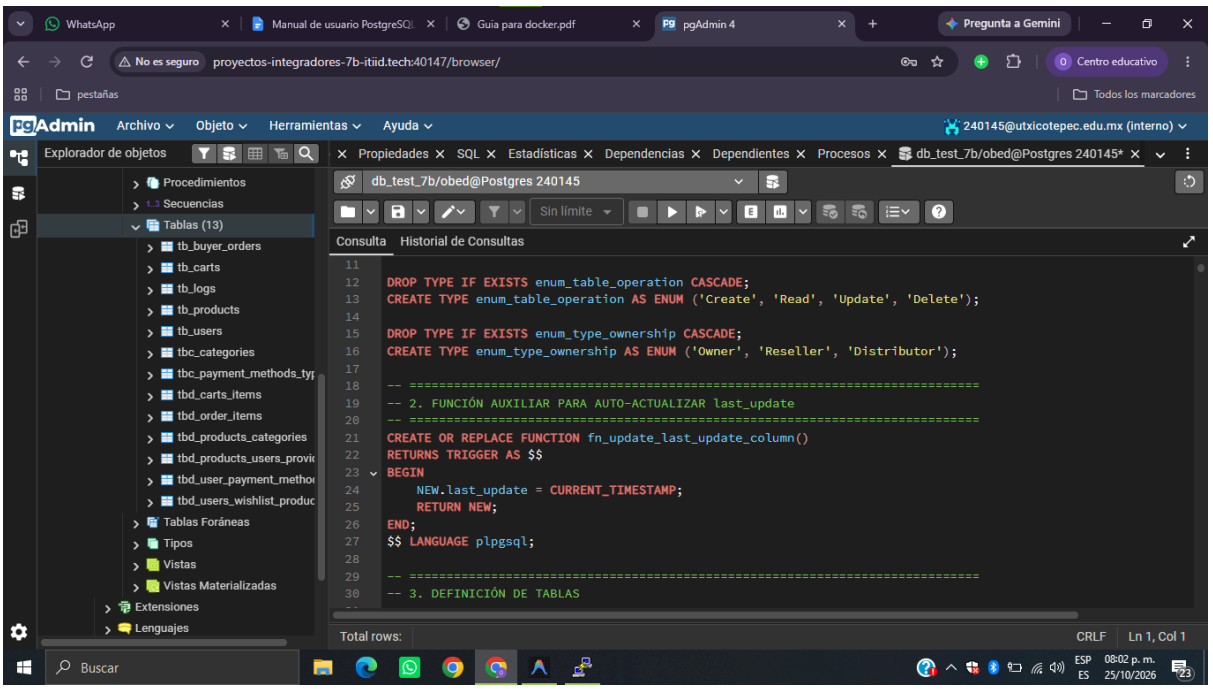


entraremos a la pestaña de Query tools y aqui pegamos nuestro script de nuestra base de datos que e cambio de MySQL a PostgreSQL

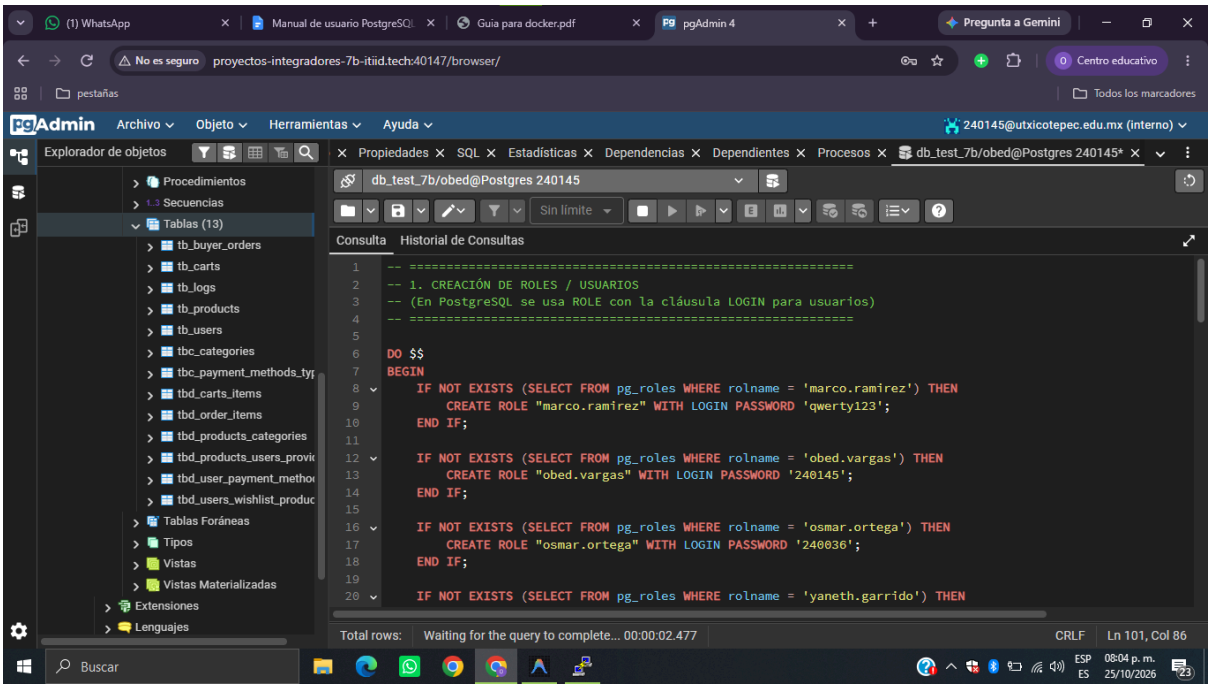


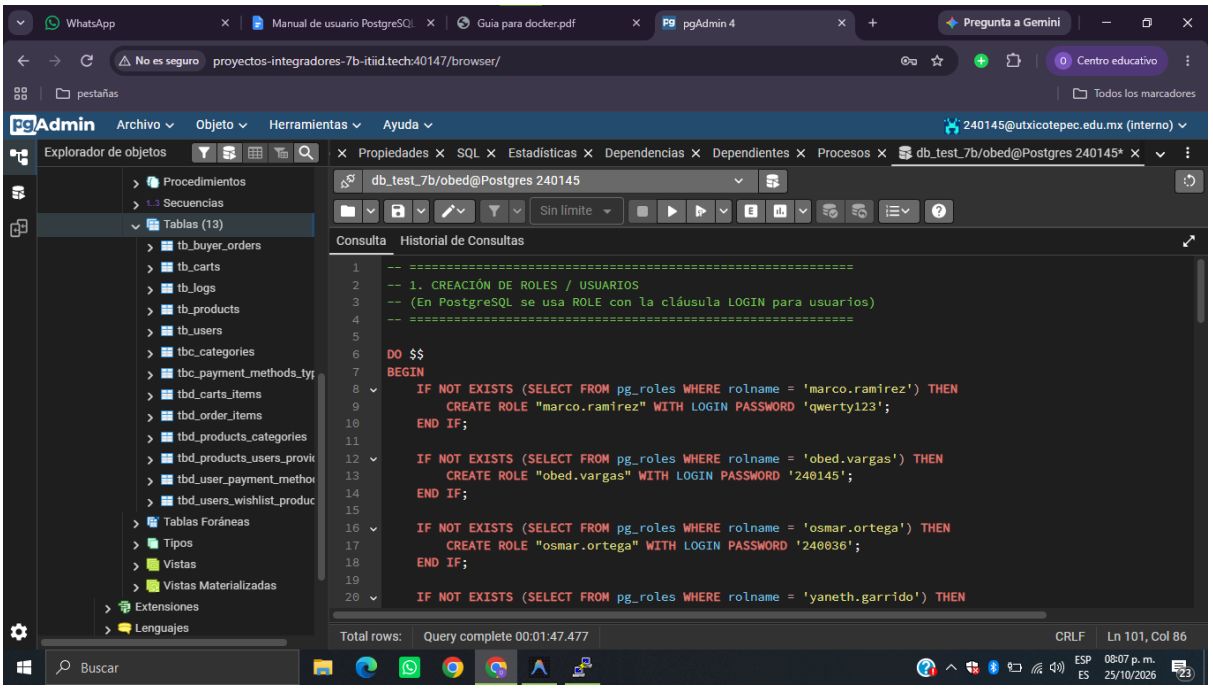



pgAdmin 4
proyectos-integradores-7b-itiid.tech:40147/browser/
db_test_7b/obed@Postgres 240145
Consulta retornó exitosamente en 16 secs 226 msec.
Total rows:
Query complete 00:00:16.226
CRLF
Ln 779, Col 1


pgAdmin 4
proyectos-integradores-7b-itiid.tech:40147/browser/
db_test_7b/obed@Postgres 240145
Total rows:
CRLF
Ln 1, Col 1

5, Corremos el archivo de remote_users_privileges_creation.sql





10. Los 10 ejercicios.